

Spring Boot

Cron Schedulers

`@Scheduled · @EnableScheduling · expresiones cron`

1. Introducción

Los Cron Schedulers en Spring Boot permiten ejecutar métodos automáticamente en momentos o intervalos definidos. Se integran de forma nativa sin dependencias adicionales.

Casos de uso comunes:

- Limpieza periódica de registros en base de datos
- Generación de reportes automáticos
- Sincronización con servicios o APIs externas
- Envío de notificaciones o correos programados

2. Configuración Inicial

2.1 Habilitar el scheduling en la clase principal

Agrega `@EnableScheduling` sobre la clase anotada con `@SpringBootApplication`:

```
@SpringBootApplication
@EnableScheduling
public class MiAplicacion {

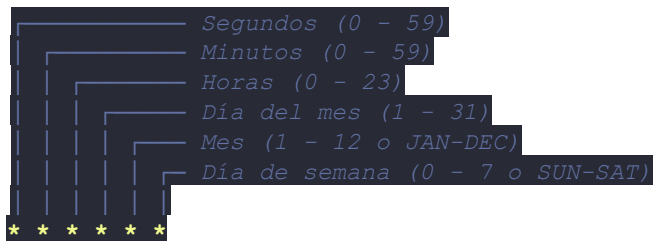
    public static void main(String[] args) {
        SpringApplication.run(MiAplicacion.class, args);
    }
}
```



Sin `@EnableScheduling` ningún método con `@Scheduled` se ejecutará, aunque esté correctamente anotado.

3. Anatomía de una Expresión Cron

Spring Boot usa expresiones cron de 6 campos (a diferencia del cron de Unix que tiene 5):



Caracteres especiales:

Carácter	Significado	Ejemplo
*	Cualquier valor	* en horas = cada hora
?	Sin valor específico	? en día-semana (cuando se usa día-mes)
-	Rango de valores	1-5 = lunes a viernes
,	Lista de valores	1,3,5 = lun, mié, vie
/	Incremento / paso	0/15 = cada 15 unidades desde 0
L	Último (Last)	L en día-mes = último día del mes
W	Día hábil más cercano (Weekday)	15W = día hábil más cercano al 15
#	N-ésimo día de la semana	2#3 = tercer martes del mes

4. Expresiones Cron más Usadas

Expresión	Cuándo se ejecuta
0 * * * * *	Cada minuto (al segundo 0)
0 */5 * * * *	Cada 5 minutos
0 */15 * * * *	Cada 15 minutos
0 */30 * * * *	Cada 30 minutos

<code>0 0 * * * *</code>	Cada hora en punto
<code>0 0 8 * * *</code>	Todos los días a las 8:00 AM
<code>0 0 8 * * MON-FRI</code>	Lunes a viernes a las 8:00 AM
<code>0 0 0 * * *</code>	Cada día a medianoche
<code>0 0 0 1 * *</code>	El primer día de cada mes
<code>0 0 0 L * *</code>	El último día de cada mes
<code>0 0 9 * * MON</code>	Cada lunes a las 9:00 AM
<code>0 0 0 1 1 *</code>	El 1° de enero a medianoche
<code>0 0/30 9-17 * * *</code>	Cada 30 min entre 9AM y 5PM

5. Uso de @Scheduled

5.1 Estructura base del componente

Siguiendo el patrón del proyecto: @Log4j2, @Component, @RequiredArgsConstructor:

```
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import lombok.extern.log4j.Log4j2;

@Log4j2
@Component
public class TareasProgramadas {

    // Se ejecuta todos los días a las 6:00 AM
    @Scheduled(cron = "0 0 6 * * *")
    public void tareaMatutina() {
        log.info("Tarea matutina ejecutada: {}", LocalDateTime.now());
    }

    // Se ejecuta cada 30 minutos
    @Scheduled(cron = "0 */30 * * * *")
    public void verificarEstado() {
        log.info("Verificando estado del sistema...");
    }
}
```

5.2 Expresión cron externalizada en application.yml

Es una buena práctica no hardcodear la expresión; así se puede cambiar sin recompilar:

```
# application.yml
app:
  cron:
    reporte: "0 0 8 * * MON-FRI"
    limpieza: "0 0 0 1 * *"
    sincronizacion: "0 */15 * * * *"
```

```
@Log4j2
@Component
public class TareasConfigurables {

    @Scheduled(cron = "${app.cron.reporte}", zone = "America/Bogota")
    public void generarReporteDiario() {
        log.info("Generando reporte: {}", LocalDateTime.now());
    }

    @Scheduled(cron = "${app.cron.limpieza}", zone = "America/Bogota")
    public void limpiarDatosAntiguos() {
        log.info("Limpiando registros antiguos...");
    }

    @Scheduled(cron = "${app.cron.sincronizacion}", zone = "America/Bogota")
    public void sincronizarConApi() {
        log.info("Sincronizando con servicio externo...");
    }
}
```

6. Zona Horaria

Por defecto Spring usa la zona horaria del servidor. Especificarla explícitamente evita errores en producción:

```
@Log4j2
@Component
public class TareasConZona {

    // Zona horaria Colombia — recomendado especificarla siempre
    @Scheduled(cron = "${app.cron.reporteMensual}", zone = "America/Bogota")
    public void reporteMensual() {
        log.info("Reporte mensual iniciado: {}", LocalDateTime.now());
    }

    // UTC para consistencia en ambientes cloud
```

```

@Scheduled(cron = "${app.cron.syncUtc}", zone = "UTC")
public void sincronizacionUtc() {
    log.info("Sincronización UTC ejecutada");
}
}
# application.yml
app:
  cron:
    reporteMensual: "0 0 7 1 * *"
    syncUtc: "0 0 12 * * *"

```



Usa siempre identificadores IANA (America/Bogota, Europe/Madrid, UTC) en vez de abreviaciones como COT o EST, que pueden ser ambiguas entre plataformas.

7. Manejo de Errores

Una excepción no controlada en un `@Scheduled` silencia todas las ejecuciones futuras de esa tarea. Siempre envuelve la lógica en `try/catch` siguiendo el mismo patrón del proyecto:

```

@Log4j2
@Component
public class TareasConManejoErrores {

    /**
     * Proceso nocturno de cierre contable.
     * Se ejecuta de lunes a viernes a la 1:00 AM hora Colombia.
     */
    @Scheduled(cron = "${app.cron.cierreDiario}", zone = "America/Bogota")
    public void cierreDiario() {
        try {
            log.info("Iniciando cierre diario: {}", LocalDateTime.now());
            ejecutarCierre();
            log.info("Cierre diario finalizado correctamente");
        } catch (Exception e) {
            log.error("Error en cierre diario: {}", e.getMessage(), e);
        }
    }

    private void ejecutarCierre() {
        // lógica del negocio aquí
    }
}

```



Si no capturas la excepción, Spring cancela el scheduler de ese método permanentemente hasta el próximo reinicio de la aplicación.

8. Con Inyección de Dependencias

Usando el mismo patrón del proyecto: @RequiredArgsConstructor + final para inyección por constructor:

```
@Log4j2
@Component
@RequiredArgsConstructor
public class ReporteScheduler {

    /** Servicio que contiene la lógica del reporte */
    private final ReporteService reporteService;

    /**
     * Genera el reporte de ventas cada día a las 7:00 AM.
     */
    @Scheduled(cron = "${app.cron.reporteVentas}", zone = "America/Bogota")
    public void reporteVentasDiario() {
        try {
            log.info("Iniciando reporte de ventas: {}", LocalDateTime.now());
            reporteService.generarReporteVentas();
            log.info("Reporte de ventas generado correctamente");
        } catch (Exception e) {
            log.error("Error al generar reporte de ventas: {}", e.getMessage(), e);
        }
    }

    /**
     * Limpia registros con más de 90 días el primer día de cada mes.
     */
    @Scheduled(cron = "${app.cron.limpiezaMensual}", zone = "America/Bogota")
    public void limpiezaMensual() {
        try {
            log.info("Iniciando limpieza mensual: {}", LocalDateTime.now());
            LocalDateTime limite = LocalDateTime.now().minusDays(90);
            reporteService.eliminarRegistrosAnterioresA(limite);
            log.info("Limpieza mensual finalizada");
        } catch (Exception e) {
            log.error("Error en limpieza mensual: {}", e.getMessage(), e);
        }
    }
}
```

```
# application.yml
app:
  cron:
    reporteVentas: "0 0 7 * * *"
    limpiezaMensual: "0 0 2 1 * *"
```

9. Buenas Prácticas

1. Externaliza siempre la expresión cron en application.yml para cambiarla sin recompilar.
2. Captura excepciones con try/catch dentro de cada método `@Scheduled`.
3. Especifica la zona horaria con el parámetro `zone` usando identificadores IANA.
4. Registra inicio y fin de cada tarea con `log.info` e incluye `log.error` en el catch.
5. Documenta cada tarea con JavaDoc indicando su propósito y frecuencia.
6. Sigue el mismo patrón del proyecto: `@Log4j2`, `@Component`, `@RequiredArgsConstructor`.
7. Agrupa las tareas relacionadas en un mismo `@Component` (ej: `ReporteScheduler`, `LimpiezaScheduler`).
8. Usa nombres descriptivos en el yaml: `app.cron.reporteVentas` es mejor que `app.cron.tarea1`.

10. Referencia Rápida

Elemento	Para qué sirve
<code>@EnableScheduling</code>	Activa el motor de scheduling en la aplicación
<code>@Scheduled(cron = "...")</code>	Programa una tarea con expresión cron de 6 campos
<code>@Scheduled(cron = "\${prop}")</code>	Expresión cron leída desde application.yml
<code>zone = "America/Bogota"</code>	Fija la zona horaria de la expresión cron
<code>try / catch + log.error</code>	Evita silenciar ejecuciones futuras ante errores
<code>@RequiredArgsConstructor</code>	Inyección de dependencias por constructor (Lombok)
<code>@Log4j2</code>	Logger automático provisto por Lombok